SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	A. Mounish	**Reg. No.:**	192472273
Section / Batch:	CSE AI	**Date:**	16\7\2026

Code of Conduct

I A. MOUNISH / 192472273 *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

1. Experiment: Resume Information Extraction Using Regular Expressions in Python

AIM:

To design and implement a Python-based Resume Information Extraction System using Regular Expressions to automatically extract candidate details such as name, email, mobile number, skills, and experience, and shortlist eligible candidates.

ALGORITHM:

1. **Start the program.**
2. **Import the re module.**
3. **Read the resume text.**
4. **Extract the candidate name using Regular Expression.**
5. **Extract email address using a suitable pattern.**
6. **Extract mobile number using a suitable pattern.**
7. **Search for technical skills (Python, Java, SQL, Machine Learning, NLP).**
8. **Extract years of experience from the resume.**
9. **Generate a structured summary of the candidate profile.**
10. **Check eligibility:**
    - **Experience ≥ 2 years**
    - **Python skill available**
11. **Display candidate details and eligibility status.**
12. **Stop the program.**

PYTHON PROGRAM

```
import

# Sample Resume Text
resume = """
Name: John Smith
Email: johnsmith@gmail.com
Mobile: 9876543210

Skills: Python, SQL, Machine Learning, NLP

Experience: 3 years
"""

# Extract Name
name = re.search(r"Name:\s*([A-Za-z ]+)", resume)
name = name.group(1) if name else "Not Found"

# Extract Email
email = re.search(r'[\w\.-]+@[\w\.-]+\.\w+', resume)
```

```
email = email.group() if email else "Not Found"

# Extract Mobile Number
mobile = re.search(r'\b\d{10}\b', resume)
mobile = mobile.group() if mobile else "Not Found"

# Extract Skills
skill_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills_found = []

for skill in skill_list:
    if re.search(skill, resume, re.IGNORECASE):
        skills_found.append(skill)

# Extract Experience
exp = re.search(r'(\d+)\s*years?', resume, re.IGNORECASE)
experience = int(exp.group(1)) if exp else 0

# Generate Summary
print("----- Candidate Profile Summary -----")
print("Name:", name)
print("Email:", email)
print("Mobile:", mobile)
print("Skills:", ", ".join(skills_found))
print("Experience:", experience, "Years")

# Eligibility Check
if experience >= 2 and "Python" in skills_found:
    print("\nStatus: Eligible for Shortlisting")
else:
    print("\nStatus: Not Eligible")
```

SAMPLE INPUT

Name: John Smith
Email: johnsmith@gmail.com
Mobile: 9876543210

Skills: Python, SQL, Machine Learning, NLP

Experience: 3 years

SAMPLE OUTPUT

----- Candidate Profile Summary -----

Name: John Smith

Email: johnsmith@gmail.com

Mobile: 9876543210

Skills: Python, SQL, Machine Learning, NLP

Experience: 3 Years

Status: Eligible for Shortlisting

RESULT

Thus, the Resume Information Extraction System was successfully implemented using Python Regular Expressions to extract candidate details and shortlist eligible candidates based on experience and Python skill requirements.

---

2. Experiment: Product Search System Using Regular Expressions in Python

AIM

To design and implement a Python-based Product Search System using Regular Expressions for exact, prefix, suffix, partial, and case-insensitive keyword matching.

\

ALGORITHM

1. **Start the program.**
2. **Import the re module.**
3. **Create a list of product names.**
4. **Accept a search keyword.**
5. **Perform:**
   - **Exact keyword search.**
   - **Prefix-based search.**
   - **Suffix-based search.**
   - **Partial keyword search.**
   - **Case-insensitive search.**
6. **Store all matching products.**
7. **Display the matching product names.**
8. **Count the number of matches for each search type.**
9. **Generate and display a search report.**
10. **Stop the program.**

PYTHON PROGRAM

```
import re

# Product List
products = [
    "Laptop",
    "Laptop Bag",
    "Gaming Laptop",
    "Smartphone",
    "Phone Case",
    "Bluetooth Speaker",
    "Wireless Mouse",
    "Mouse Pad",
    "Smart Watch",
    "Watch Charger"
]

keyword = "Laptop"

# Exact Search
exact_match = [p for p in products if re.fullmatch(keyword, p, re.IGNORECASE)]

# Prefix Search
prefix_match = [p for p in products if re.search(r'^' + keyword, p, re.IGNORECASE)]

# Suffix Search
suffix_match = [p for p in products if re.search(keyword + r'$', p, re.IGNORECASE)]

# Partial Search
partial_match = [p for p in products if re.search(keyword, p, re.IGNORECASE)]

# Case-Insensitive Search
case_insensitive_match = [p for p in products if re.search(keyword, p, re.IGNORECASE)]

# Display Results
print("EXACT SEARCH:")
print(exact_match)

print("\nPREFIX SEARCH:")
print(prefix_match)

print("\nSUFFIX SEARCH:")
```

```
print(suffix_match)

print("\nPARTIAL SEARCH:")
print(partial_match)

print("\nCASE-INSENSITIVE SEARCH:")
print(case_insensitive_match)

# Report
print("\n----- SEARCH REPORT -----")
print("Exact Matches:", len(exact_match))
print("Prefix Matches:", len(prefix_match))
print("Suffix Matches:", len(suffix_match))
print("Partial Matches:", len(partial_match))
print("Case-Insensitive Matches:", len(case_insensitive_match))
```

SAMPLE INPUT

Keyword = Laptop

SAMPLE OUTPUT

```
EXACT SEARCH:
['Laptop']

PREFIX SEARCH:
['Laptop', 'Laptop Bag']

SUFFIX SEARCH:
['Laptop', 'Gaming Laptop']

PARTIAL SEARCH:
['Laptop', 'Laptop Bag', 'Gaming Laptop']

CASE-INSENSITIVE SEARCH:
['Laptop', 'Laptop Bag', 'Gaming Laptop']

----- SEARCH REPORT -----
Exact Matches: 1
Prefix Matches: 2
Suffix Matches: 2
Partial Matches: 3
Case-Insensitive Matches: 3
```

RESULT

Thus, the Product Search System was successfully implemented using Python Regular Expressions to perform exact, prefix, suffix, partial, and case-insensitive searches, and the matching products along with the search report were displayed successfully.

---

3. Experiment: Student Registration Validation System Using Regular Expressions

AIM

To design and implement a Python-based Student Registration Validation System using Regular Expressions to validate student details before course enrollment.

ALGORITHM

1. **Start the program.**
2. **Import the re module.**
3. **Read the student register number.**
4. **Validate the register number using a Regular Expression.**
5. **Read the institutional email address.**
6. **Validate the email format using a Regular Expression.**
7. **Read the course code.**
8. **Validate the course code using a Regular Expression.**
9. **Read the semester information.**
10. **Validate the semester value using a Regular Expression.**
11. **Read the mobile number.**
12. **Validate the mobile number using a Regular Expression.**
13. **Display validation messages for each field.**
14. **If all fields are valid, display "Registration Successful".**
15. **Otherwise, display "Registration Failed".**
16. **Stop the program.**

PYTHON PROGRAM

```
import re

# Sample Student Details
register_no = "23CS101"
email = "student@university.edu"
course_code = "CS501"
semester = "5"
mobile = "9876543210"

status = True

# Register Number Validation
```

```
if re.fullmatch(r'\d{2}[A-Z]{2}\d{3}', register_no):
    print("Register Number: Valid")
else:
    print("Register Number: Invalid")
    status = False

# Email Validation
if re.fullmatch(r'[a-zA-Z0-9._%+-]+@university\.edu', email):
    print("Email Address: Valid")
else:
    print("Email Address: Invalid")
    status = False

# Course Code Validation
if re.fullmatch(r'[A-Z]{2}\d{3}', course_code):
    print("Course Code: Valid")
else:
    print("Course Code: Invalid")
    status = False

# Semester Validation
if re.fullmatch(r'[1-8]', semester):
    print("Semester: Valid")
else:
    print("Semester: Invalid")
    status = False

# Mobile Number Validation
if re.fullmatch(r'[6-9]\d{9}', mobile):
    print("Mobile Number: Valid")
else:
    print("Mobile Number: Invalid")
    status = False

# Final Registration Status
print("\n----- REGISTRATION STATUS REPORT -----")

if status:
    print("Registration Successful")
else:
    print("Registration Failed")
```

SAMPLE INPUT

Register Number : 23CS101

Email Address : student@university.edu
Course Code : CS501
Semester : 5
Mobile Number : 9876543210

SAMPLE OUTPUT

Register Number: Valid
Email Address: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid

----- REGISTRATION STATUS REPORT -----
Registration Successful

RESULT

Thus, the Student Registration Validation System was successfully implemented using Python Regular Expressions to validate the register number, institutional email address, course code, semester information, and mobile number, and generate the final registration status report.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____